

Experiment 'p_4_5_analyse_floats_seq' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_4_5_analyse_floats_seq

Natural language statement: Write a method that takes a sequence of floatingpoint numbers and returns:
- the average of the values. - the smallest of the values. - the largest of the values. - the range, that is the difference between the smallest and largest.

Method signature: p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values) / |values| as real)`
- `forall v :: v in values ==> min <= v`
- `forall v :: v in values ==> max >= v`
- `range == max - min`
- `exists v :: v in values && min == v`
- `exists v :: v in values && max == v`

Requires

- `0 < |values|`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a sequence of floatingpoint numbers and returns: - the average
→ of the values. - the smallest of the values. - the largest of the values. - the
→ range, that is the difference between the smallest and largest.

The signature should be:

p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real,
→ range: real)

The method should respect the following contract:

requires 0 < |values|, ensures average == (sum(values) / |values| as real), ensures
→ forall v :: v in values ==> min <= v, ensures forall v :: v in values ==> max >= v
→ , ensures range == max - min, ensures exists v :: v in values && min == v, ensures
→ exists v :: v in values && max == v

The contract uses the following dafny code:

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmfXWRJi8rVfUEisnqGXaT038NUd4

Model: gpt-5.5-2026-04-23

Created at: 1780491794

Finish reason: stop

Usage: 624 tokens in, and 9146 tokens out

```
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
```

```

range := max - min;

assert values == [values[0]];
assert values[0] in values;

forall v | v in values
  ensures min <= v
{
  assert v == values[0];
}

forall v | v in values
  ensures max >= v
{
  assert v == values[0];
}

assert exists v :: v in values && min == v;
assert exists v :: v in values && max == v;
} else {
  assert 1 < |values|;
  var tail := values[1..];
  assert 0 < |tail|;
  assert |values| == |tail| + 1;
  assert values == [values[0]] + tail;
  assert forall v :: (v in values) <==> (v == values[0] || v in tail);

  var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(tail);
  assert tailRange == tailMax - tailMin;

  var tailLength := |tail| as real;
  assert tailLength != 0.0;
  assert (|values| as real) == tailLength + 1.0;

  average := (values[0] + tailAverage * tailLength) / (|values| as real);

  if values[0] <= tailMin {
    min := values[0];
    assert values[0] in values;
    assert exists v :: v in values && min == v;
  } else {
    min := tailMin;
    ghost var w: real :| w in tail && tailMin == w;
    assert w in values;
    assert exists v :: v in values && min == v;
  }

  if values[0] >= tailMax {
    max := values[0];
    assert values[0] in values;
    assert exists v :: v in values && max == v;
  }
}

```

```

} else {
  max := tailMax;
  ghost var w: real :| w in tail && tailMax == w;
  assert w in values;
  assert exists v :: v in values && max == v;
}

forall v | v in values
  ensures min <= v
{
  if v == values[0] {
    if values[0] <= tailMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert tailMin < values[0];
    }
  } else {
    assert v in tail;
    assert tailMin <= v;
    if values[0] <= tailMin {
      assert min == values[0];
      assert values[0] <= v;
    } else {
      assert min == tailMin;
    }
  }
}

forall v | v in values
  ensures max >= v
{
  if v == values[0] {
    if values[0] >= tailMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert tailMax > values[0];
    }
  } else {
    assert v in tail;
    assert tailMax >= v;
    if values[0] >= tailMax {
      assert max == values[0];
      assert values[0] >= v;
    } else {
      assert max == tailMax;
    }
  }
}

```

```

    range := max - min;
  }
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_4_5_analyse_floats_seq/program.dfy(15,0): Error: a postcondition could not
  ↳ be proved on this return path

```

```

|
15 | {
|   ^

```

```

problem_data/p_4_5_analyse_floats_seq/program.dfy(9,17): Related location: this is the
  ↳ postcondition that could not be proved

```

```

|
9 | ensures average == (sum(values) / |values| as real)
|   ^^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmfaOLkiRTAZvF8dgtNorfoAqYN5I

Model: gpt-5.5-2026-04-23

Created at: 1780491972

Finish reason: stop

Usage: 1594 tokens in, and 4558 tokens out

```

{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := max - min;

    assert values == [values[0]];
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);
  }
}

```

```

assert values[0] in values;

forall v | v in values
  ensures min <= v
{
  assert v == values[0];
}

forall v | v in values
  ensures max >= v
{
  assert v == values[0];
}

assert exists v :: v in values && min == v;
assert exists v :: v in values && max == v;
} else {
  assert 1 < |values|;
  var tail := values[1..];
  assert 0 < |tail|;
  assert |values| == |tail| + 1;
  assert values == [values[0]] + tail;
  assert forall v :: (v in values) <==> (v == values[0] || v in tail);

  var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(tail);
  assert tailAverage == sum(tail) / (|tail| as real);
  assert tailRange == tailMax - tailMin;

  var tailLength := |tail| as real;
  assert tailLength == (|tail| as real);
  assert tailLength != 0.0;
  assert (|values| as real) == tailLength + 1.0;

  average := (values[0] + tailAverage * tailLength) / (|values| as real);

  calc {
    tailAverage * tailLength;
    == {
      assert tailAverage == sum(tail) / tailLength;
    }
    (sum(tail) / tailLength) * tailLength;
    == {
      assert tailLength != 0.0;
    }
    sum(tail);
  }

  assert values[1..] == tail;
  assert sum(values) == values[0] + sum(values[1..]);
  assert sum(values) == values[0] + sum(tail);

```

```

calc {
  average;
  == {
  }
  (values[0] + tailAverage * tailLength) / (|values| as real);
  == {
    assert tailAverage * tailLength == sum(tail);
  }
  (values[0] + sum(tail)) / (|values| as real);
  == {
    assert sum(values) == values[0] + sum(tail);
  }
  sum(values) / (|values| as real);
}

if values[0] <= tailMin {
  min := values[0];
  assert values[0] in values;
  assert exists v :: v in values && min == v;
} else {
  min := tailMin;
  ghost var w: real :| w in tail && tailMin == w;
  assert w in values;
  assert exists v :: v in values && min == v;
}

if values[0] >= tailMax {
  max := values[0];
  assert values[0] in values;
  assert exists v :: v in values && max == v;
} else {
  max := tailMax;
  ghost var w: real :| w in tail && tailMax == w;
  assert w in values;
  assert exists v :: v in values && max == v;
}

forall v | v in values
  ensures min <= v
{
  if v == values[0] {
    if values[0] <= tailMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert tailMin < values[0];
    }
  } else {
    assert v in tail;
    assert tailMin <= v;
    if values[0] <= tailMin {

```

```

        assert min == values[0];
        assert values[0] <= v;
    } else {
        assert min == tailMin;
    }
}

forall v | v in values
  ensures max >= v
{
  if v == values[0] {
    if values[0] >= tailMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert tailMax > values[0];
    }
  } else {
    assert v in tail;
    assert tailMax >= v;
    if values[0] >= tailMax {
      assert max == values[0];
      assert values[0] >= v;
    } else {
      assert max == tailMax;
    }
  }
}

range := max - min;
}

```

Final Program

```

function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max
  ↪ : real, range: real)
  requires 0 < |values|
  ensures average == (sum(values) / |values| as real)
  ensures forall v :: v in values ==> min <= v
  ensures forall v :: v in values ==> max >= v
  ensures range == max - min
  ensures exists v :: v in values && min == v

```



```

        ensures exists v :: v in values && max == v
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := max - min;

    assert values == [values[0]];
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);

    assert values[0] in values;

    forall v | v in values
      ensures min <= v
    {
      assert v == values[0];
    }

    forall v | v in values
      ensures max >= v
    {
      assert v == values[0];
    }

    assert exists v :: v in values && min == v;
    assert exists v :: v in values && max == v;
  } else {
    assert 1 < |values|;
    var tail := values[1..];
    assert 0 < |tail|;
    assert |values| == |tail| + 1;
    assert values == [values[0]] + tail;
    assert forall v :: (v in values) <==> (v == values[0] || v in tail);

    var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(tail);
    assert tailAverage == sum(tail) / (|tail| as real);
    assert tailRange == tailMax - tailMin;

    var tailLength := |tail| as real;
    assert tailLength == (|tail| as real);
    assert tailLength != 0.0;
    assert (|values| as real) == tailLength + 1.0;

    average := (values[0] + tailAverage * tailLength) / (|values| as real);
  }
}

```

```

calc {
  tailAverage * tailLength;
  == {
    assert tailAverage == sum(tail) / tailLength;
  }
  (sum(tail) / tailLength) * tailLength;
  == {
    assert tailLength != 0.0;
  }
  sum(tail);
}

assert values[1..] == tail;
assert sum(values) == values[0] + sum(values[1..]);
assert sum(values) == values[0] + sum(tail);

calc {
  average;
  == {
  }
  (values[0] + tailAverage * tailLength) / (|values| as real);
  == {
    assert tailAverage * tailLength == sum(tail);
  }
  (values[0] + sum(tail)) / (|values| as real);
  == {
    assert sum(values) == values[0] + sum(tail);
  }
  sum(values) / (|values| as real);
}

if values[0] <= tailMin {
  min := values[0];
  assert values[0] in values;
  assert exists v :: v in values && min == v;
} else {
  min := tailMin;
  ghost var w: real :| w in tail && tailMin == w;
  assert w in values;
  assert exists v :: v in values && min == v;
}

if values[0] >= tailMax {
  max := values[0];
  assert values[0] in values;
  assert exists v :: v in values && max == v;
} else {
  max := tailMax;
  ghost var w: real :| w in tail && tailMax == w;
  assert w in values;
  assert exists v :: v in values && max == v;
}

```

```

}

forall v | v in values
  ensures min <= v
{
  if v == values[0] {
    if values[0] <= tailMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert tailMin < values[0];
    }
  } else {
    assert v in tail;
    assert tailMin <= v;
    if values[0] <= tailMin {
      assert min == values[0];
      assert values[0] <= v;
    } else {
      assert min == tailMin;
    }
  }
}

forall v | v in values
  ensures max >= v
{
  if v == values[0] {
    if values[0] >= tailMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert tailMax > values[0];
    }
  } else {
    assert v in tail;
    assert tailMax >= v;
    if values[0] >= tailMax {
      assert max == values[0];
      assert values[0] >= v;
    } else {
      assert max == tailMax;
    }
  }
}

range := max - min;
}

```

Total Token Usage

Input tokens: 2218

Output tokens: 13704

Reasoning tokens: 11445

Sum of 'total tokens': 15922

Experiment Timings

Overall Experiment started at 1780491793847, ended at 1780492060642, lasting 266795ms (266.80 seconds)

Iteration #1 started at 1780491793847, ended at 1780491972601, lasting 178754ms (178.75 seconds)

Iteration #2 started at 1780491972602, ended at 1780492060642, lasting 88040ms (88.04 seconds)